Student Name : Orla Fitzgerald
Project Repo URL : https://github.com/oohlah/happy_pipes
Render Website Address : https://happy-pipes.onrender.com

Grade Band	IoT Solution	Networking/IoT Technologies	Combined Knowledge	Communication
Base	**Sensor-focused IoT system** using simulated temperature and humidity data in Cisco Packet Tracer. **Data is sent to a Raspberry Pi,** which displays system state using Sense HAT LEDs and captures images using the Pi Camera.	Cisco simulated devices send UDP data (connectionless, unreliable) telemetry transfer in fast succession, so no concern if some data is lost. This a basic one-direction, simple network topology from one device to another. Use of the Pi camera and sense hat LEDS integrates hardware interaction with the incoming network data.	**Programming:** Python reads sensor data from simulated devices and the Pi. I've used threshold checks, and control outputs such as LEDs. **Database:** Readings are saved locally in a JSON file. **Computer systems:** The Pi hardware is used to display system status via LEDs and capture environmental images with the camera.	DONE
Good	**Raspberry Pi** acts as an edge device receiving telemetry from Cisco simulation via UDP. **A listener service** processes the data and publishes structured **JSON** messages to an **MQTT** broker.**Basic analytics applied** (dew point calculation) before forwarding data.	**Networking technologies:UDP** used for telemetry from Cisco Packet Tracer to the **listener service. MQTT** used to distribute structured sensor data to other processes. **Listener service bridges t**he simulated network and messaging layer.	**Programming:** Python handles UDP reception from Cisco simulation, parses JSON telemetry, calculates dew point, updates state, and publishes structured messages via MQTT. Threading used to allow listener service to run continuously without blocking. **Structured Data Messages:** Local JSON file stores the current sensor state (temperature, humidity, dew point, timestamps, chart/image placeholders), ensures structured and persistent storage. **Hardware interaction:** Sense HAT LEDs indicate system status (green for normal, red for threshold alerts). Logging real-time data to console for feedback.	DONE

			Basic html dashboard service with camera image updates.	
Excellent	**Multi-node system:** Raspberry Pi sends sensor data → Flask API collects and stores it → Web dashboard shows current readings, historical charts, and camera images. The backend stores **historical readings** in CSV files and uses them to show **trends over time**, not just current values. Camera images captured on the Pi are uploaded to **Cloudinary**, enabling cloud-based image storage and remote access. The system presents both **live and historical environment data**, including charts and camera images, allowing users to understand how conditions change over time. Status indicators (LEDs and dashboard alerts) are driven by **rules based on trends and accumulated data**.	**MQTT** is used as a lightweight messaging protocol to send sensor updates from the edge device to backend services in real time. A **Flask-based backend service** acts as an API layer, exposing current readings and historical data to the dashboard using structured HTTP endpoints.**Cloudinary** provides cloud-based image hosting for camera captures, decoupling image storage from the Raspberry Pi. **Separation of concerns** between edge device (data capture), Messaging layer (MQTT), Backend/API (storage and processing), UI/dashboard (visualisation). Automatic chart generation and dashboard refresh provide **live updates** without manual intervention	**Programming:**Python is used across multiple parts of the system, including background data handling, a Flask-based API, and automated chart generation. Different tasks run at the same time so the system can continue processing data while also serving the web dashboard. **Data management:** Historical sensor readings are stored in CSV files, allowing the system to work with data collected over time rather than only current values. JSON is used to represent the latest system state in a structured and readable format. **Visualisation and UI:** Matplotlib is used to generate charts from historical sensor data, which are displayed on a web dashboard alongside current readings and camera images. The dashboard updates automatically to reflect new data**. Cloud integration with Cloudinary** is used to manage camera images and chart image in the cloud. **Blynk** enables remote monitoring through an external IoT platform. **Hardware components** (Sense HAT LEDs and Pi camera) are now **driven by backend logic informed by historical**	DONE - all resources added to GitHub

			data and rules, linking physical feedback to system-wide state rather than immediate sensor readings alone.	
Outstanding	A complete IoT system for monitoring environmental conditions using both simulated sensors and a Raspberry Pi. The system collects data, processes it, stores it in the cloud, and displays it on web and mobile interfaces. Suitable for real-world use cases such as smart homes, building monitoring, or environmental tracking. Designed in a way that reflects how real IoT systems are built, making it suitable as a portfolio project.	Uses multiple networking technologies including **UDP**, **MQTT**, **HTTP**, and cloud-based IoT platforms. **Blynk** Cloud allows live sensor data to be viewed remotely on a mobile device. **Cloudinary** is used to store and serve camera images captured by the Raspberry Pi. **A deployed web dashboard (Render)** allows access to the system outside the local network. Shows effective use of cloud services to improve access, scalability, and usability.	**Independent learning:** CSV-based historical data storage and analysis were implemented beyond taught material. Matplotlib was learned independently and used to create clear data visualisations. Built on course material by extending the Packet tracer simulation to include multiple interacting simulated devices, including a hearing device responding to environmental conditions. **Programming and systems integration -**Python connects networking, cloud services, backend processing, and visualisation across multiple services.	DONE - The project combines hardware, networking, cloud platforms, data storage, and user interfaces. It is publicly accessible through a deployed website and a well-documented GitHub repository with diagrams and a demo video.

Additional Comments: